3.5.6 Common filters

Tags are sometimes used to render a specific value, like with {% now %} or {% url %}, but are mostly used for control flow, like {% if %} and {% for %}. Often, you want to perform an action on some data, modifying how it will be displayed. Consider a `datetime` object in Python: it stores date and time information, but when you want to print it out, you have a large number of choices for formatting the display. By changing the arguments to the `.strftime()` method, you change how the date is shown. Filters in Django templates are a generic way of doing this kind of action on all kinds of data.

Filters apply inside double-brace surrounded variables with the filter's change applied to the variable before it is rendered. You use a filter through the pipe (|) operator within the double braces. The input to a filter is the contents of the variable, while its output is an action applied to the data.

Two commonly used filters are `| upper` and `| lower`. These behave like the `str.upper()` and `str.lower()` calls, changing the case of the corresponding string. The template `{{ word | upper }}` results in rendering the contents of `word` after changing all its letters to upper case.

The following are some other common filters:

- `{{ num | pluralize }}`—Returns `s` if the value of `num` is greater than 1
- `{{ words | first }}`—Returns the first item in a `list`
- `{{ words | last }}`—Returns the last item in a `list`

The `{{ | pluralize }}` filter is useful for composing counting sentences. For example, the template text `You have {{ num }} message{{ num | pluralize }}` gets rendered as "You have 3 messages." when `num` contains three. The `{{ | first }}` and `{{ | last }}` filters are handy when you need items out of a list. You can always use `{{ words.0 }}` to also get the first, but as the template language doesn't support negative indexing, the only way to get the last item is with this filter.

Like tags, filters can take arguments. You pass arguments to a filter using the colon (:) character. The `| join` filter works like Python's `str.join()` method, taking a `list` and constructing a string from the result. This filter takes an argument specifying the separator between the joined items, most commonly a comma (,). The template `{{ words | join:"," }}` is equivalent to the Python `",".join(words)`.

You've now seen how to create `Template` objects, how to render them using a `Context`, and an overview of some key tags and filters. Doing all of this in a view would make your Python code rather unwieldy. Almost every view you write is going to want to render a template to return some HTML to your user. Because this is such a frequent activity, Django provides a shortcut method, called `render()`. This shortcut loads a template from a file, creates a `Context` object based on a dictionary, and returns an `HttpResponse` object, all in a single line of code.

3.6 Using render() in a view

Each time someone visits a page on *RiffMates* your view will return some HTML to their browser. The vast majority of your views will do a bit of logic processing and then